

Query, Key, Value Projections in Self-Attention

A full worked numerical example — every matrix multiplication done by hand, $d_{\text{model}} = 2$, $d_k = 2$, three tokens.

1. Setup

Sentence: **"cats eat fish"**. Each token starts as a plain 2-dimensional embedding (made up for illustration — in a real model these come from a learned embedding table).

X (shape 3×2 , one row per token)

	dim 0	dim 1
cats	1	0
eat	0	1
fish	1	1

Token embedding matrix X.

Three weight matrices are learned during training and reused for every token: W_Q , W_K , W_V , each of shape $(d_{\text{model}}, d_k) = (2, 2)$.

W_Q

1	0
1	1

W_K

0	1
1	0

W_V

1	2
0	1

Learned weight matrices W_Q , W_K , W_V (values chosen for illustration).

Row-vector convention used throughout: for a row $[a, b]$ multiplied by a 2×2 matrix $\begin{bmatrix} p & q \\ r & s \end{bmatrix}$, the result is $[a \cdot p + b \cdot r, a \cdot q + b \cdot s]$.

2. Step 1 — Compute $Q = X \cdot W_Q$

Every token's embedding is projected through the same W_Q . This produces each token's **query** vector — a representation of what that token is "looking for" in the rest of the sequence.

$$q_{\text{cats}} = [1, 0] \cdot W_Q = [1 \cdot 1 + 0 \cdot 1, 1 \cdot 0 + 0 \cdot 1] = [1, 0]$$

$$q_{\text{eat}} = [0, 1] \cdot W_Q = [0 \cdot 1 + 1 \cdot 1, 0 \cdot 0 + 1 \cdot 1] = [1, 1]$$

$$q_{\text{fish}} = [1, 1] \cdot W_Q = [1 \cdot 1 + 1 \cdot 1, 1 \cdot 0 + 1 \cdot 1] = [2, 1]$$

Q (shape 3×2)

cats	1	0
------	---	---

eat	1	1
fish	2	1

Query matrix Q — one query vector per token.

3. Step 2 — Compute $K = X \cdot W_K$

The same tokens are projected through W_K to produce each token's **key** vector — a representation of what that token "advertises" to be matched against other tokens' queries.

$$k_{\text{cats}} = [1, 0] \cdot W_K = [1 \cdot 0 + 0 \cdot 1, 1 \cdot 1 + 0 \cdot 0] = [0, 1]$$

$$k_{\text{eat}} = [0, 1] \cdot W_K = [0 \cdot 0 + 1 \cdot 1, 0 \cdot 1 + 1 \cdot 0] = [1, 0]$$

$$k_{\text{fish}} = [1, 1] \cdot W_K = [0 + 1, 1 + 0] = [1, 1]$$

K (shape 3×2)

cats	0	1
eat	1	0
fish	1	1

Key matrix K.

4. Step 3 — Compute $V = X \cdot W_V$

The tokens are projected a third time through W_V to produce each token's **value** vector — the actual content that gets passed forward if this token ends up being attended to.

$$v_{\text{cats}} = [1, 0] \cdot W_V = [1, 2]$$

$$v_{\text{eat}} = [0, 1] \cdot W_V = [0, 1]$$

$$v_{\text{fish}} = [1, 1] \cdot W_V = [1, 3]$$

V (shape 3×2)

cats	1	2
eat	0	1
fish	1	3

Value matrix V.

5. Step 4 — Raw Attention Scores = $Q \cdot K^T$

Every query is dot-producted against every key. Entry $\text{scores}[i, j]$ measures how strongly token i 's query matches token j 's key.

$q_{\text{cats}} \cdot k_{\text{cats}} = [1, 0] \cdot [0, 1] = 0$ $q_{\text{cats}} \cdot k_{\text{eat}} = [1, 0] \cdot [1, 0] = 1$ $q_{\text{cats}} \cdot k_{\text{fish}} = [1, 0] \cdot [1, 1] = 1$
 $q_{\text{eat}} \cdot k_{\text{cats}} = [1, 1] \cdot [0, 1] = 1$ $q_{\text{eat}} \cdot k_{\text{eat}} = [1, 1] \cdot [1, 0] = 1$ $q_{\text{eat}} \cdot k_{\text{fish}} = [1, 1] \cdot [1, 1] = 2$
 $q_{\text{fish}} \cdot k_{\text{cats}} = [2, 1] \cdot [0, 1] = 1$ $q_{\text{fish}} \cdot k_{\text{eat}} = [2, 1] \cdot [1, 0] = 2$ $q_{\text{fish}} \cdot k_{\text{fish}} = [2, 1] \cdot [1, 1] = 3$

scores (shape 3×3, row = query token, column = key token)

	cats	eat	fish
cats	0	1	1
eat	1	1	2
fish	1	2	3

$\text{scores}[\text{fish}, \text{fish}] = 3$ is the highest value in the matrix — fish's query matches fish's own key more than anything else's (self-attention).

6. Step 5 — Scale by $\sqrt{d_k}$

Every entry is divided by $\sqrt{d_k} = \sqrt{2} \approx 1.414$. This keeps the dot-product magnitudes from growing with dimensionality, which would otherwise push softmax into regions with vanishingly small gradients.

scaled scores

	cats	eat	fish
cats	0.000	0.707	0.707
eat	0.707	0.707	1.414
fish	0.707	1.414	2.121

7. Step 6 — Row-wise Softmax

$\text{softmax}(x)_i = \exp(x_i) / \sum_j \exp(x_j)$, applied independently to each row so every row sums to 1.

Row 0 — "cats"

$\exp(0) = 1.000$, $\exp(0.707) = 2.028$, $\exp(0.707) = 2.028$ $\text{sum} = 5.056$
 $\text{weights} = [1.000/5.056, 2.028/5.056, 2.028/5.056] \approx [0.198, 0.401, 0.401]$

Row 1 — "eat"

$\exp(0.707) = 2.028$, $\exp(0.707) = 2.028$, $\exp(1.414) = 4.113$ $\text{sum} = 8.169$
weights $\approx [0.248, 0.248, 0.503]$

Row 2 — "fish"

$\exp(0.707) = 2.028$, $\exp(1.414) = 4.113$, $\exp(2.121) = 8.343$ $\text{sum} = 14.484$
weights $\approx [0.140, 0.284, 0.576]$

attention weights (each row sums to 1.0)

	cats	eat	fish
cats	0.198	0.401	0.401
eat	0.248	0.248	0.503
fish	0.140	0.284	0.576

"fish" allocates 57.6% of its attention to itself, 28.4% to "eat", and 14.0% to "cats".

8. Step 7 — Output = Weights · V

Each output row is a weighted sum of every value vector, using that row's attention weights as the mixing coefficients.

cats' new representation

$$\begin{aligned}\text{out_cats} &= 0.198 \cdot [1, 2] + 0.401 \cdot [0, 1] + 0.401 \cdot [1, 3] \\ &= [0.198, 0.396] + [0, 0.401] + [0.401, 1.203] = [0.599, 2.000]\end{aligned}$$

eat's new representation

$$\begin{aligned}\text{out_eat} &= 0.248 \cdot [1, 2] + 0.248 \cdot [0, 1] + 0.503 \cdot [1, 3] \\ &= [0.248, 0.496] + [0, 0.248] + [0.503, 1.509] = [0.751, 2.253]\end{aligned}$$

fish's new representation

$$\begin{aligned}\text{out_fish} &= 0.140 \cdot [1, 2] + 0.284 \cdot [0, 1] + 0.576 \cdot [1, 3] \\ &= [0.140, 0.280] + [0, 0.284] + [0.576, 1.728] = [0.716, 2.292]\end{aligned}$$

Final output (shape 3×2)

cats	0.599	2.000
eat	0.751	2.253
fish	0.716	2.292

Each token's original embedding, now replaced by a context-aware blend of the sequence's value vectors.

9. Summary

Each token started as a 2-D vector. Projecting every token through the shared W_Q , W_K , W_V matrices produced three views: what each token is asking about (Q), what each token advertises (K), and what each token would contribute (V). Dot-producting Q against K^T gave a similarity score for every (query, key) pair. Scaling by $\sqrt{d_k}$ kept the softmax numerically well-behaved. Softmax turned scores into a probability distribution per row. Multiplying those weights by V mixed each token's output as a weighted blend of every value vector — so every output token is now a context-aware representation, not just its original embedding.

In a full transformer this block runs in parallel across multiple heads, each with its own independent W_Q , W_K , W_V . The per-head outputs are concatenated and passed through a final linear layer that projects back to d_{model} .

Worked example: self-attention Q/K/V projections, $d_{\text{model}} = 2$, $d_k = 2$, 3 tokens.